

Documentação do site — Catálogo de Bebidas

Visão geral

Aplicação web para catálogo de bebidas com visualização de produtos, filtro por categorias, busca, abertura de modal de reserva e registro de reservas com atualização de estoque.

Tecnologias utilizadas

- Next.js
- React
- TypeScript
- Supabase
- react-hook-form
- Lucide React
- Tailwind CSS

Estrutura principal do projeto

- `src/`

- `app/`

- `page.tsx`

Página principal do catálogo, com busca, filtros, listagem de produtos e abertura do modal de reserva.

- `api/`

- `reservas/`

- `route.ts`

Endpoint responsável por listar reservas e criar novas reservas.

- `components/`

- `CatalogCard.tsx`

Card de exibição de produto no catálogo.

- `ReservaModal.tsx`

Modal de preenchimento e confirmação da reserva.

- `Toast.tsx`

Componente de feedback visual de sucesso ou erro.

- `lib/`

- `supabase.ts`

Cliente Supabase usado no front-end e no back-end.

- `types/`

- `index.ts`

Tipos e interfaces da aplicação.

- `next.config.js`

Configuração do Next.js, incluindo liberação de domínios remotos para imagens.

- `package.json`

Dependências, scripts e configurações do projeto.

- `.env.local`

Variáveis de ambiente para conexão com o Supabase e autenticação administrativa.

Fluxo da aplicação

- O usuário acessa a página principal.

- O sistema carrega:

- produtos

- categorias

- O usuário pode:

- pesquisar por nome

- filtrar por categoria
- abrir o modal de reserva
- Ao confirmar a reserva:
 - os dados do formulário são enviados para a API
 - a reserva é salva no banco
 - o estoque do produto é atualizado
 - o usuário recebe feedback visual

Componentes principais

page . tsx

Responsável por:

- buscar produtos e categorias;
- aplicar filtros;
- renderizar os cards de produtos;
- abrir o modal de reserva;
- enviar a reserva para a API;
- exibir toast de retorno.

ReservaModal . tsx

Responsável por:

- exibir dados do produto selecionado;
- receber nome, telefone e quantidade;
- validar os campos;
- salvar nome e telefone no `localStorage`;

- disparar a confirmação da reserva.

route.ts

Responsável por:

- GET /api/reservas
- lista reservas
- exige senha administrativa no header `x-admin-password`
- POST /api/reservas
- cria uma nova reserva
- valida campos obrigatórios
- verifica estoque
- insere a reserva
- atualiza o estoque do produto

Integração com Supabase

A aplicação utiliza dois clientes:

- **supabase**
 - usado em contextos públicos
 - conectado com a chave anônima
- **supabaseAdmin**
 - usado no servidor
 - conectado com a chave de serviço
 - permite operações administrativas como inserir reservas e atualizar estoque

Variáveis de ambiente

- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_ANON_KEY
- SUPABASE_SERVICE_ROLE_KEY
- ADMIN_PASSWORD

Estrutura do banco de dados

Tabela categorias

Armazena as categorias dos produtos.

Campo	Tipo	Descrição
id	integer	Identificador da categoria
nome	text	Nome da categoria

Tabela produtos

Armazena os produtos exibidos no catálogo.

Campo	Tipo	Descrição
id	text	Identificador do produto
nome	text	Nome do produto
imagem_url	text	URL da imagem do produto
estoque	integer	Quantidade disponível
preco	numeric	Preço do produto
descricao	text	Descrição do produto
ativo	boolean	Indica se o produto está ativo
criado_em	timestamp	Data de criação
categoria_id	integer	Chave estrangeira para categorias.id

Tabela reservas

Armazena as reservas realizadas pelos usuários.

Campo	Tipo	Descrição
id	text	Identificador da reserva
produto_id	text	Chave estrangeira para produtos.id
nome_comprador	text	Nome de quem fez a reserva
telefone	text	Telefone de contato
quantidade	integer	Quantidade reservada
status	text	Situação da reserva
criado_em	timestamp	Data de criação

Relacionamentos do banco

- `produtos.categoria_id` 'categorias.id
- `reservas.produto_id` 'produtos.id

Regras de negócio

- Um produto só pode ser reservado se houver estoque suficiente.
- Após uma reserva confirmada:
- a reserva é gravada na tabela `reservas`
- o estoque do produto é reduzido
- O nome e o telefone informados na reserva podem ser reaproveitados via `localStorage`.
- A listagem de reservas administrativas exige autenticação por senha.

Configuração de imagens remotas

O projeto permite carregar imagens de domínios externos, inclusive de marketplaces e CDNs, por meio da configuração de imagens remotas do Next.js.

Observações de funcionamento

- A página principal depende da API de produtos e categorias para montar o catálogo.
- O modal de reserva depende do produto selecionado.
- O retorno da API de reservas precisa ser JSON válido para evitar erro no front-end.
- O estoque exibido no catálogo deve refletir o valor salvo no banco.